LINQ Child Nutrition Program (CNP)

Foundational Architecture

Program-Wide Bedrock + Phase 1 Vertical

This document defines the foundational architecture that underpins the entire multi-phase CNP program and shows exactly which slice rides on top of it for Phase 1. Foundation is built once; verticals — Phase 1's SNP application-to-approval, Phase 2's Claims and Federal Reports and AI layer, Phase 3's Food Distribution — plug into it without modification.

Document version: 0.1 | Date: 30 June 2026

1. The Four-Layer Mental Model

The CNP platform is best understood as four horizontal layers stacked on top of one another. Lower layers are program-wide foundation — built once, used by every state and every phase. Upper layers are program-specific or phase-specific verticals that ride on the foundation. The boundary between "foundation" and "vertical" is sharp: nothing in the foundation knows about a specific program, state, or feature.

Layer 4	VERTICALS — phase-specific feature slices Phase 1: SNP Application-to-Approval (one state) Phase 2: Claims, Federal Reports, all programs, all states, AI Layer Phase 3: Food Distribution, CACFP IMP activation, Offline Review
Layer 3	DOMAINS — program-specific business logic on top of the foundation Program domains (SNP, CACFP, DCH, SFSP, FFVP, SSFO, CEP) Cross-program domains (Claims, Eligibility, Compliance, Accounting) Each domain plugs into the engines via named hooks and configuration rows
Layer 2	ENGINES — configurable, data-driven, federally guardrailed Workflow Engine · Rules Engine · Form Engine · Review Instrument Engine Federal Report Engine (Phase 2) · VCA Instrument Engine (Phase 2) All engines share the IsFederal enforcement contract and read from cnp_global
Layer 1	FOUNDATION — built once, used by everything above Multi-tenant data plane (cnp_global + cnp_ops + cnp_ext + Snowflake) Identity (Keycloak OIDC) · Authorisation (ICnpPermissionService) Configuration (ICnpConfigService) · Secrets (ISecretProvider) Audit (append-only) · Notifications · Background Jobs (Hangfire) Frontend shell (Angular + snackpaq + Formly) · Integration adapter contracts Federal-guardrail pattern (IsFederal) · DevOps & observability

Phase 1 builds Layer 1 in full, builds a subset of Layer 2 sufficient to run an application workflow (Workflow Engine + Form Engine), builds one Layer 3 program domain (SNP) at minimum-viable scope, and builds one Layer 4 vertical (Application-to-Approval). Phases 2 and 3 add domains and verticals without modifying the foundation.

2. Why This Shape — Four Principles That Force the Layering

1. One codebase, many state footprints. State variation cannot live in code branches, so it must live in either (a) configuration rows, (b) feature flags, or (c) pluggable adapters — all three are foundation concerns. The Layer-1 foundation provides the homes; the Layer-3 domains and Layer-4 verticals fill them.
2. Federal guardrails are structural. The IsFederal=true marker must be enforced at write time across configuration, RBAC, workflow, and form definitions. Because the marker is cross-cutting, the enforcement lives in Layer 1 (the configuration service, the permission service, the workflow service) — never in a domain.
3. Frozen-at-approval semantics. When an application is approved, the active rates and rules are snapshotted onto a ClaimLaws record. Because this freeze pattern is needed by every program (SNP, CACFP, SFSP, …), the snapshot mechanism is Layer-1 / Layer-2 — never per-program code.
4. Database is the tenant boundary. A separate Aurora cluster per state means there is no TenantId column on operational tables; the tenant scoping is solved once in Layer 1 (ICnpContext + connection resolution) and used unmodified by everything above.

3. Layer 1 — Foundation (Program-Wide Bedrock)

Below are the seventeen foundation components. For each: its purpose, the contract it exposes to upper layers, and the Phase-1 scope (what gets built in the first 16 weeks).

3.1 Data Plane

Foundation component	Purpose	Contract exposed to upper layers	Phase-1 scope
F1 — cnp_global (shared federal layer)	One Aurora Postgres instance holding all federal-truth data: rates, programs, lookup codes, workflow definitions, rule definitions, form definitions, RBAC catalogue, configuration keys, query templates. Read by every state.	Postgres read-only access via ICnpContext.GlobalDb	Built. Phase-1 subset of the ~387 tables seeded: programs, workflow defs, form defs, RBAC catalogue, config keys for SNP application.
F2 — cnp_ops (per-state operational)	One Aurora Postgres instance per state — sponsors, sites, applications, claims, reviews, audit. Database itself is the tenant boundary; no TenantId column on operational tables.	Postgres read/write via ICnpContext.StateDb	Built for the chosen demo state. Per-state migration runner in place for Phase-2 multi-state.
F3 — cnp_ext (per-state extensions)	Schema in the same Aurora cluster as cnp_ops for genuinely state-specific tables (Ohio Budget+42, Wyoming gratefdp). Never referenced from shared platform code.	Per-state extension schema	Scaffolded but unused in Phase 1.
F4 — Snowflake (analytics warehouse)	Read-only mirror of cnp_ops via CDC pipeline. All reports, analytics, AI features read from Snowflake — operational Aurora is never queried for analytics. Tenant isolation by row-access policy.	Snowflake views + row-access policies	Account provisioned. CDC pipeline scaffolded; no Phase-1 read traffic.

3.2 Tenant Resolution & Isolation

Foundation component	Purpose	Contract exposed to upper layers	Phase-1 scope
F5 — Tenant resolution chain	Subdomain (e.g. mn.linqcnp.com) → YARP reverse-proxy classifies tenant → ICnpContext is built per request with the	ICnpContext { GlobalDb, StateDb, TenantCode } — injected per request	Built and exercised by the demo state's subdomain.

Foundation component	Purpose	Contract exposed to upper layers	Phase-1 scope
	right cnp_global + cnp_ops connection strings.		
F6 — Tenant Registry	Provisioning record in cnp_global tracking when each state was onboarded, what version of the platform they run, and which features are active for them.	TenantRegistry table + admin API (Phase 2)	Single row seeded for the demo state.

3.3 Identity, Authorisation, Secrets

Foundation component	Purpose	Contract exposed to upper layers	Phase-1 scope
F7 — Keycloak (OIDC broker) Who are you?/ Authentication:JWT Token	One Keycloak deployment on ECS Fargate brokers all state IdPs. Per-tenant realms federate ClassLink / Okta / Azure AD (Phase 1 OIDC-native states) and SAML/ADFS (Phase 2 SAML states).	OIDC endpoints; JWT carries roles + tenant scope (never raw permissions)	Deployed; one demo-state realm configured.
F8 — ICnpPermissionService (authZ) What are you allowed to do?/ Authorization:	Authorisation is owned exclusively here. Three-layer override: RolePermission → TenantRoleOverride → UserPermissionOverride. Explicit Revoke at any layer beats Grant below. IsFederal=true permissions cannot be revoked.	ICnpPermissionService.Check(user, permission, scope) — sole accessor	Built; Phase-1 SECURITY domain RolePermissions seeded (~25 permissions).
F9 — ISecretProvider How do we safely access passwords and credentials?	Single interface fronts all secrets. App code only sees reference paths like cnp/{env}/{tenant|global}/{category}/{name} — never values. Backed by AWS Secrets Manager. Offshore principals structurally denied access to production paths.	ISecretProvider.Get(path) → SecureString	Built; production interface contract enforced even though Phase 1 has no production.

3.4 Configuration & Federal Guardrail

Foundation component	Purpose	Contract exposed to upper layers	Phase-1 scope
F10 — ICnpConfigService	~3,000 typed, bounded, effective-dated configuration keys (CnpDefaults). Three-layer resolution: Platform → Program → Tenant. Most-specific wins via COALESCE. Three-tier cache (Immediate / Session / Nightly).	ICnpConfigService.GetValue<T>(key) — sole accessor; SetValue gated by IsFederal check	Built; Phase-1 subset of CnpDefaults seeded (SNP application keys). 3-tier cache wired (Redis).
F11 — Federal guardrail (IsFederal pattern)	Cross-cutting: every CnpDefault, RolePermission, WorkflowTransition, FormField, and ComplianceRuleDef carries an IsFederal column. Override attempts on IsFederal=true rows are blocked at write time and audit-logged.	Write-time check in every Layer-1 service; PR-blocking assertion in tests	Built and enforced across Config, RBAC, Workflow services.
F12 — LaunchDarkly (feature flags)	Toggles whole capabilities per tenant with no deploy. ~47 module-level flags at full scale. Evaluated by Angular route guards on the frontend and by feature-flag attributes on the backend.	LaunchDarkly SDK + ICnpFeatureFlagService wrapper	Integrated; ~5 flags wired for Phase 1.

3.5 Audit, Notifications, Background Jobs

Foundation component	Purpose	Contract exposed to upper layers	Phase-1 scope
F13 — Audit Service	Append-only capture of every state-changing event. Three structures: AuditLogs (event header), EntitySnapshots (full prior-state JSONB), FieldChangeLog (field-level diffs). Point-in-time queryable.	IAuditService.Emit(event) — auto-wired into all Layer-2 engines	Built; 100% capture for Phase-1 surface. Audit Timeline query API live.
F14 — Notification Service	Outbound only — no business rules. Email templates in cnp_global, broadcast and in-app messaging via Intercom (Phase 2). Triggered by workflow actions, not by domain code.	INotificationService.Send(template, recipients, context)	Built; minimal templates wired for Phase-1 application-state-change emails.
F15 — Hangfire (background jobs)	Per-tenant scheduled work. Phase-1 jobs: nightly synthetic-data refresh, audit-log rollup. Phase-2 jobs: PY rollover, DC nightly loads, FNA report generation.	IRecurringJobManager + Hangfire job attributes	Built; 2 Phase-1 jobs registered.

3.6 Integration & Frontend

Foundation component	Purpose	Contract exposed to upper layers	Phase-1 scope
F16 — Integration adapter contracts	Four interfaces define every state-specific integration: IDcFeedAdapter, IAccountingExporter, IEdFiAdapter, ILicenseAdapter. Keyed DI registration per state code. No if/else branches on state anywhere in shared code.	Four interfaces in cnp_platform; per-state implementations in cnp_extensions.{state}.dll resolved at startup	Contracts defined; one stub IDcFeedAdapter (returns canned data) for the demo state.
F17 — Frontend shell	Multi-tenant Angular 17+ SPA. @linq/snackpaq design system with CSS-variable brand tokens. Angular Formly for server-driven forms. Keycloak OIDC integration. LaunchDarkly SDK. URL pattern /{programYear}/...	Application shell module + ICnpContext (browser) + per-tenant branding pipeline	Built. Application list + application detail + audit timeline + one config-admin screen for the speed demo.

4. Layer 2 — Engines

Engines are configurable, data-driven services that all share the same federal-guardrail contract. They live above the foundation and below the domain code. Engines have zero knowledge of any specific program or state.

Engine	How it works (foundation-level description)	Phase scope
E1 — Workflow Engine	Data-driven state machine. WorkflowEntityTypes, WorkflowStates, WorkflowTransitions, WorkflowConditions, WorkflowActions stored as rows in cnp_global. TenantTransitionOverrides in cnp_ops. Conditions and actions resolve to named hooks (ConditionEvaluator / ActionExecutor classes) registered per entity type. IsFederal guardrails on every row.	Phase 1: built. WorkflowEntityType for SponsorApplication registered. State machine: Draft → Submitted → StateReview → ReturnedForCorrection → Approved.
E2 — Rules Engine	Microsoft Rules Engine on JSONB rules stored in cnp_global, versioned and effective-dated. Rule categories: FEDERAL_IMMUTABLE / FEDERAL_CONFIGURABLE / STATE_CONFIGURABLE / TENANT_CONFIGURABLE. Federal math is excluded by design — that lives in code.	Phase 1: scaffolded; minimal usage (state-configurable application-validation rules). Full activation Phase 2.
E3 — Form Engine	Year-versioned FormDefinitions in cnp_global. Server emits FormlyFieldConfig[] consumed by Angular Formly on the frontend. Conditional fields, required/optional, hidden, validation rules — all metadata-driven.	Phase 1: built. SNP application FormDefinition seeded; multi-step Formly form renders end-to-end.
E4 — Review Instrument Engine	Powers structured questionnaires used in compliance reviews. Configurable questions, answer types, scoring, findings.	Phase 2.
E5 — Federal Report Engine	Plug-in IFederalReportBuilder per (ReportType, StateCode). Orchestrator sequences ValidateAsync → ResolveSubmissionMode → BuildPart{A/B/C/C2} → AssembleAsync. Reads from Snowflake, writes ReportPackage to S3.	Phase 2 (FNA-44 first); Phase-1 scaffolds the contracts only.
E6 — VCA Instrument Engine	Configurable scoring tool for auditing Food Service Management Company contracts.	Phase 2.

5. Layer 3 — Domains

Domains are the program-specific business logic that uses the engines and lives above them. Each domain plugs into the engines through configuration rows (workflow definitions, rules, forms) and named hooks (federal action handlers), never through code-level extension of the engines themselves.

5.1 Domain Tiering (from the customer's Domains document)

The customer's State CNP Platform Domains document defines four tiers with strict dependency rules. The foundation sits below all of these; the engines sit between the foundation and the domains.

Tier	Domains	Phase 1 coverage
Tier 1 — Authoritative	Hold legal, financial, and enforcement authority. Claims & Reimbursement, Accounting & Disbursement, Compliance & Administrative Review, Eligibility & Direct Certification.	Phase 1 partial (Sponsor + Application foundations); full in Phase 2.
Tier 2 — Supporting / Upstream	Constrain inputs to authoritative domains. Identity & Access, Configuration & Tenant, Sponsor & Site Registry, Applications & Agreements, Program Registry.	Phase 1 — full coverage of the IAM, Config, Sponsor & Site Registry, Applications surfaces.
Tier 3 — Specialist Supporting	Program domains (SNP, CACFP, DCH, SFSP, FFVP, SSFO, CEP) + platform services (Forms, Workflow, Integration, Communications, Document Library).	Phase 1 — SNP only. Other program domains added in Phase 2.
Tier 4 — Downstream / Observational	Read models and outbound services only. Reporting & Analytics (Snowflake), Audit & Observability, Notifications, Reference Data, LaunchDarkly.	Phase 1 — Audit + Notifications + LaunchDarkly + Reference Data live. Reporting/Analytics scaffolded for Phase 2.

6. Layer 4 — Phase 1 Vertical (What Actually Ships in October)

The Phase 1 vertical is the thinnest possible slice that exercises every foundation component end-to-end. It is deliberately narrow on features (one state, one program, application only — no claims, no reports) so that the foundation is built for real, not stubbed.

6.1 The Vertical, Step by Step

Here is what happens when a sponsor uses the Phase 1 system. Every step exercises one or more foundation components, named in brackets.

5. Sponsor navigates to mn.linqcnp.com (or the chosen demo state's subdomain). The platform resolves the tenant via YARP and builds an ICnpContext [F5, F6].
6. Sponsor is redirected to Keycloak for OIDC login against the demo state's identity provider [F7].
7. On callback, the JWT (roles + tenant scope) is carried in service memory. ICnpPermissionService gates every UI element [F8].
8. Sponsor sees the application list — a server-driven query of cnp_ops.{demo-state} for applications scoped to this sponsor [F2, F8].
9. Sponsor clicks "Create new application". The frontend asks the Form Engine [E3] for the SNP application FormDefinition, receives a FormlyFieldConfig[], and renders the multi-step form.
10. Sponsor saves drafts and submits. Submission invokes a workflow transition (Draft → Submitted) [E1]. The transition writes an audit row [F13] and emits a notification template [F14].
11. State caseworker logs in (same Keycloak realm, different role). Sees pending applications. Permission check on Approve is gated by ICnpPermissionService [F8].
12. Caseworker either returns for correction (a ReturnedForCorrection transition with comments — workflow action [E1] + notification [F14]) or approves.
13. On approval, the workflow engine [E1] runs the NoNDLMatch federal pre-condition hook. The hook calls the NDL adapter [F16]; a canned ConfirmedMatch blocks the approval, an audit row is written [F13], and the caseworker sees a federal-block notification [F14].
14. On a clean approval, the FreezeClaimLaws action [E1] snapshots the active rates and rules from cnp_global into a ClaimLaws row in cnp_ops [F1, F2]. The audit captures the snapshot [F13].
15. A LINQ administrator opens the Config Admin screen, changes the application deadline CnpDefault, and saves. ICnpConfigService [F10] applies the IsFederal check [F11], invalidates the Immediate-tier Redis cache, and writes the change to FieldChangeLog [F13]. Within 10 minutes, the sponsor's UI reflects the new deadline. "Speed demo" achieved.
16. At any point, anyone with the Audit.View permission [F8] opens the Audit Timeline for the application and reconstructs the full who/when/what history from EntitySnapshots and FieldChangeLog [F13].

6.2 Phase 1 Vertical — Concretely, What Code Gets Written

Area	What gets built
Domain code (SNP application)	SponsorApplication entity + repository; SNP application FormDefinition seed; NoNDLMatch hook implementation; SNP-specific workflow definition seed; SNP-specific RolePermission seeds; demo-state synthetic data.
Frontend (Phase 1 surface)	Application list screen, application detail screen (Formly-rendered), audit timeline panel, config admin screen, tenant-branding pipeline, OIDC bootstrap, route guards, layout shell.
Integration (Phase 1)	Stub NDL adapter (returns canned ConfirmedMatch / PossibleMatch / NoMatch). No DC adapters, no accounting exporters.
Tests	Characterization tests against the legacy SNP application-approval stored procedure (≥ 80% branch coverage); Playwright E2E covering the 9 functional requirements; xUnit on every Layer-1 service.

Area	What gets built
Infrastructure	Terraform for the Phase-1 environment set (Dev, Staging, Demo, UAT, per-PR ephemeral). GitHub Actions pipeline. AWS CodeDeploy. ECS Fargate services.

7. What Phase 2 and Phase 3 Add — Without Modifying the Foundation

The whole point of building the foundation correctly in Phase 1 is that Phase 2 and Phase 3 do not touch it. They plug into it.

7.1 What Phase 2 plugs in

- Six additional program domains (CACFP, DCH, SFSP, FFVP, SSFO, CEP) — each is FormDefinition rows in cnp_global, workflow-definition rows in cnp_global, RolePermission rows, and per-program hook implementations. Zero foundation change.
- Claims domain — a new WorkflowEntityType for Claim, its workflow definition rows, its hook implementations (calculation, validation, freeze-application-of-rates). The Workflow Engine [E1] runs it unchanged.
- Federal Report Engine [E5] — IFederalReportBuilder implementations per (ReportType, StateCode), all backed by the Snowflake [F4] read path scaffolded in Phase 1.
- DC Feed adapters (Indiana, Hawaii) — IDcFeedAdapter implementations per state, registered into the keyed DI slot [F16]. Adapters write to staging tables; the Eligibility domain owns the merge.
- Accounting exporters — IAccountingExporter implementations per state (USAS, CALSTARS, SCEIS, …), again keyed DI slot [F16].
- AI Layer — Compliance Risk Scoring, Program Year Change Detection, NLP Query, Anomaly Detection, Config-Assist. Reads from Snowflake [F4]; writes through the existing approval gates [F8, F10].
- Full admin UI — additional Angular routes and screens, all using the same frontend shell [F17], snackpaq, and feature-flag gates.
- Multi-region — Terraform modules already designed for multi-region; Phase 2 enables the second region and the cross-region replication.
- New states onboarded — a new state is a TenantRegistry row [F6], a Keycloak realm [F7], a cnp_ops Aurora cluster [F2], CnpDefault overrides [F10], and the four adapter implementations [F16]. No code deploy.

7.2 What Phase 3 plugs in

- Food Distribution domain — likely as cnp_ext schemas per state (mirroring Ohio's existing fdp / Wyoming's gratefdp) plus a FoodDistribution domain code module that reads/writes cnp_ext. The foundation accommodates this; no Layer 1 or Layer 2 change.
- CACFP IMP rule activation — RuleCategory promotions (STATE_CONFIGURABLE → FEDERAL_CONFIGURABLE) plus a per-state Cacfp.ImpRulesEnabled flag in CnpDefaults. Pure data and configuration change.
- Offline review checkout (if confirmed) — either a PWA capability added to the frontend shell [F17] or a ReviewCheckout API endpoint plus companion-app pattern. Architecture decision made during Phase 2.

8. Foundation Build Order — Which Pieces Get Built First

Inside the 16-week Phase 1, the foundation components are built in the order their dependents need them. The build order below maps onto the sprint plan from the Proposal.

Window	Foundation work that ships
Mobilization (Weeks 1–2)	F4 Snowflake account, F7 Keycloak install, F9 ISecretProvider, F17 Frontend skeleton, F16 four-interface contracts defined (no implementations), DevOps / observability bootstrap.
Sprint 1 (Weeks 3–4) — Foundation	F1 cnp_global schema v1, F2 cnp_ops schema v1, F5 ICnpContext, F13 Audit Service scaffolding, F11 IsFederal column on schemas, F17 frontend shell with OIDC login working.
Sprint 2 (Weeks 5–6) — Config + Workflow tables	F10 ICnpConfigService with three-layer resolution and IsFederal write-block, F12 LaunchDarkly integration, E1 Workflow Engine tables in cnp_global, F11 federal guardrail fully wired across Config + RBAC + Workflow services.
Sprint 3 (Weeks 7–8) — Workflow Execution + Rights	E1 Workflow execution engine, named hooks (ConditionEvaluator + ActionExecutor), F8 ICnpPermissionService with three-layer override, F6 TenantRegistry row for demo state, NoNDLMatch federal hook (Phase-1 stub adapter).
Sprint 4 (Weeks 9–10) — Form Engine + ClaimLaws	E3 Form Engine server-side FormlyFieldConfig generator, ClaimLaws table + FreezeClaimLaws action, F14 Notification Service minimal, F15 Hangfire jobs registered.
Sprint 5–6 (Weeks 11–14) — Frontend integration	F17 frontend application list, detail, audit timeline, config admin. Speed-demo plumbing exercised end-to-end.
Hardening (Weeks 15–16)	Security review of every foundation interface. Characterization-test baseline. Performance baseline. Onboarding test (Section 5.3 of the v0.2 proposal).

9. Foundation Acceptance Criteria

These are the criteria that prove the foundation is built, not just the Phase 1 vertical. They are required for the platform to be ready for Phase 2 to begin without rework.

17. Tenant isolation — verified by introducing a second synthetic state into cnp_ops, provisioning a TenantRegistry row, and demonstrating that a user logged into one state cannot see the other state's data even via direct API call.
18. Federal guardrail enforcement — verified by attempting to override an IsFederal=true CnpDefault, an IsFederal RolePermission, and an IsFederal WorkflowTransition; each attempt blocks at write time, returns the same error shape, and audit-logs identically.
19. Configuration speed-demo — a CnpDefault change made via the Config Admin UI is reflected in the running application within ten minutes, with the FieldChangeLog row capturing actor / before / after / timestamp.
20. ClaimLaws freeze — on application approval, the active rates from cnp_global.ClaimRateAmounts are written into a cnp_ops.ClaimLaws row; subsequent reads of the application show the snapshotted values even after the live rates change.
21. Audit completeness — every state-changing event in the prototype produces matching rows in AuditLogs, EntitySnapshots, and FieldChangeLog. A reviewer can reconstruct the full history of any application using only these three tables.
22. Secrets contract — no secret value appears in any application repository file or environment variable; ISecretProvider.Get(path) is the only access pattern; the offshore principal demonstrably cannot access production paths.
23. Adapter contracts — IDcFeedAdapter, IAccountingExporter, IEdFiAdapter, ILicenseAdapter all defined and resolvable via keyed DI; one stub IDcFeedAdapter for the demo state proves the registration pattern.
24. Migration runner — a second synthetic state can be provisioned end-to-end (cnp_ops cluster + schema + seed data + TenantRegistry row + Keycloak realm) by running the migration runner only, with no application code change.

25. Frontend shell — a second synthetic state's subdomain renders with the second state's brand tokens, OIDC realm, and feature-flag set, without a frontend code change.
26. Definition of Done (from the customer's own October Prototype doc) — the LINQ configuration team can re-execute the prototype outcome on a different demo state without a single new line of code.

10. Closing — Why This Foundation Is the Right One

The foundation described above is not invented; it is what the customer's own documents converge on. The Synopsis names it as five blocks (Platform Foundation, Engines, Configuration & Access, Integrations, Operations & Safety). The State CNP Platform Domains doc names it as four tiers with the per-state adapter pattern. The Frontend Architecture doc names it as three configuration layers. The Workflow Engine, Configuration Defaults, Rights, and Secrets Management docs each name their respective accessor contracts. The October Prototype doc names the Phase-1 slice.

What this document adds is the explicit four-layer mental model and the explicit mapping between the 17 foundation components and the Phase-1 vertical that rides on them. With this in place, the team building Phase 1 always knows whether the line of code they are about to write belongs in the foundation (and therefore must be designed for Phase 2 and beyond) or in the vertical (and therefore can be optimised for the prototype demo). That single distinction is the difference between a foundation that survives Phase 2 and a foundation that has to be rebuilt.